Alubys AI Development System

Reference Guide

Version 1.0 — May 2026

A reusable AI operating system for project work.
Persistent memory, structured workflows, and session protocols
that let any AI work on your project with full context, every session.

Contents

1. What This System Is

The Alubys AI Development System is a reusable AI operating system for project work. It gives AI agents durable project context through structured memory files, workflow modes, mode-specific prompts, and installer scripts.

The goal is simple: a new AI session should be able to read the project memory and continue work without relying on chat history.

The Problem It Solves

Every AI session normally starts cold. You re-explain the architecture, re-describe constraints, re-clarify decisions made weeks ago. This system fixes that: project intelligence lives in structured files that any AI reads in minutes and picks up exactly where the last session left off.

The Three Layers

Layer	What It Is	How to Use It
Operating System	`MASTER_SYSTEM_PROMPT.md`	AI reads from disk, or paste into chat
Memory Files	`.agent/` directory in each project	AI reads at startup, updates throughout
Mode Prompts	Prompt files in your project root	AI reads from disk, or paste when switching phases

> **Key principle:** The intelligence lives in the files, not in the AI's chat history. Switch AI models anytime — the new AI reads the same files and gets the same context.

2. Prerequisites & Installation

What You Need

Mac / Linux

- An AI assistant (Claude, Codex, GPT-4, Gemini, or any capable AI)
- Terminal (Mac Terminal, iTerm2, or VSCode integrated terminal)
- The alubys-ai-dev-system repository cloned once to a permanent location
- Git (for version control — optional but recommended)

Windows

- An AI assistant (Claude, Codex, GPT-4, Gemini, or any capable AI)
- PowerShell 5.1 or later (built into Windows 10 and 11 — no install needed)
- The alubys-ai-dev-system repository cloned once to a permanent location
- Git for Windows (required for cloning — https://git-scm.com/download/win)

Clone Once to a Permanent Location

The alubys-ai-dev-system repo is a tool you install once, not something you clone into every project. Choose a permanent home on your machine and clone it there. You will reference this location when setting up each new project.

```
# Mac / Linux — clone ONCE to a permanent location
mkdir -p ~/Tools
cd ~/Tools
git clone https://github.com/AlubysLLC/Alubys-AI-Dev-System.git

# You now have: ~/Tools/Alubys-AI-Dev-System/
# Leave it here. Do not clone it again for each project.
```

```
# Windows (PowerShell) — clone ONCE to a permanent location
mkdir C:\Tools
cd C:\Tools
git clone https://github.com/AlubysLLC/Alubys-AI-Dev-System.git

# You now have: C:\Tools\Alubys-AI-Dev-System\
# Leave it here. Do not clone it again for each project.
```

What "Paste This File" Means

For file-aware AI tools (Claude Code, Cursor, Codex), the AI reads prompt files directly from disk — no pasting needed.

For chat-only tools (Claude.ai, ChatGPT, Gemini), you will see instructions like "paste MASTER_SYSTEM_PROMPT.md." This means: open the file in any text editor, select all the text, copy it, and paste it directly into the AI chat input.

> **Tip:** Open file in editor → Select All (Cmd+A on Mac, Ctrl+A on Windows/Linux) → Copy (Cmd+C / Ctrl+C) → Paste into AI chat (Cmd+V / Ctrl+V). Always paste the raw text — do not attach the file.

3. Core Principles

- **Continuity over chat history**: project intelligence lives in files.
- **Mode discipline**: brainstorming, planning, execution, and reflection are separate workflows.
- **Lean memory**: memory should be accurate, concise, and useful to the next session.
- **Templates as source of truth**: generated project files come from `templates/`.
- **Small safe changes**: evolve projects incrementally.
- **Secrets stay local**: tracked files may reference secret names, never real values.

4. Core File Structure

After installation, the system files live in your project root alongside your code:

```
your-project/
■■■ _PROMPTS/ ← all AI workflow prompt files
■ ■■■ MASTER_SYSTEM_PROMPT.md ← full AI operating doctrine
■ ■■■ START_NEW_PROJECT_PROMPT.md
■ ■■■ ONBOARD_EXISTING_PROJECT_PROMPT.md
■ ■■■ BRAINSTORMING_MODE_PROMPT.md
■ ■■■ PLANNING_MODE_PROMPT.md
■ ■■■ EXECUTION_MODE_PROMPT.md
■ ■■■ REFLECTION_MODE_PROMPT.md
■■■ AGENTS.md ← canonical AI instructions
■■■ CLAUDE.md ← thin pointer to AGENTS.md (for Claude)
■■■ .agent/ ← all project memory lives here
```

The `.agent/` memory directory:

```
.agent/
■■■ .gitignore
■■■ secrets.example.md
■■■ memory/
■■■ planning/
■■■ tasks/
■■■ sessions/
■■■ reflections/
```

`AGENTS.md` is the canonical instruction file for all AI agents. `CLAUDE.md` is intentionally thin — it tells Claude to read and follow `AGENTS.md`. Other AI-specific instruction files follow the same pattern.

5. Workflow Modes

The system enforces four distinct phases of work. Stay in the current mode until you explicitly advance.

MODE 1 — BRAINSTORMING

Use when: Starting a new project or major feature. Problem space not yet defined.

Do: Ask open questions. Explore approaches. Surface constraints and risks. Define MVP scope.

Don't: Write production code. Make final architectural decisions. Rush to planning.

Key files: `brainstorming.md` `constraints.md` `risks.md`

MODE 2 — PLANNING

Use when: Brainstorming is complete and the problem is well understood.

Do: Make architectural decisions with rationale. Define MVP precisely. Create roadmap and backlog.

Don't: Begin implementation. Leave decisions ambiguous. Over-engineer for hypothetical scale.

Key files: `product-specification.md` `architecture-proposals.md` `roadmap.md` `backlog.md`

MODE 3 — EXECUTION

Use when: Planning is approved and it is time to build.

Do: Work in small reviewable increments. Update memory files continuously. Surface unexpected issues.

Don't: Rewrite large sections without authorization. Skip memory updates. Change architecture unilaterally.

Key files: `active-context.md` `changelog.md` `in-progress.md` `completed.md`

MODE 4 — REFLECTION

Use when: End of a phase, after a difficult session, or on a weekly cadence.

Do: Identify recurring mistakes. Surface workflow inefficiencies. Compress stale memory files.

Don't: Make code changes. Add features. Use this as a status update session.

Key files: `lessons-learned.md` `weekly-review.md` `agent-improvements.md`

> **How to switch modes:** For file-aware AI agents, say "Enter [mode name] mode." For chat-only tools, open the corresponding file in `_PROMPTS/`, copy all the text, and paste it into the AI chat.

6. Startup Protocol

Every AI session begins the same way, whether it is the first session or the fiftieth. This takes 30 seconds.

1 **Open a new session with your AI assistant**
Start a fresh chat or open your project in a file-aware AI tool.

2 **Load the system prompt**
For file-aware agents, the AI reads `AGENTS.md` and `_PROMPTS/MASTER_SYSTEM_PROMPT.md` automatically. For chat-only tools, paste the full text of `_PROMPTS/MASTER_SYSTEM_PROMPT.md`.

3 **Say: "Read the startup files and tell me where we are."**
The AI reads 5 key files from `.agent/` and returns a brief status summary.

4 **Confirm or correct the summary**
If the summary is accurate, proceed. If anything is wrong, correct it before any work begins.

5 **Work begins**
The AI is now fully oriented with current project state.

Files the AI Reads at Startup

File	Purpose
`.agent/memory/active-context.md`	Current project state — the most important file
`.agent/tasks/in-progress.md`	What is actively being worked on right now
`.agent/tasks/roadmap.md`	Where the project is going
`.agent/memory/lessons-learned.md`	Rules and patterns to honor this session
`.agent/memory/decisions.md`	Architecture decisions already made

7. Approval Modes

Approval modes let each project decide how much autonomy the AI should have.

Mode	Behavior
Plan-First Mode	The AI presents a complete plan and waits for explicit approval before changing files, moving/deleting files, running commands, executing scripts, installing dependencies, or implementing changes.
Standard Mode	The AI proceeds with safe scoped work after orientation, but asks before destructive, risky, broad, or external-impact actions.
Autonomous Mode	The AI proceeds through implementation and verification within the requested scope, while still asking before destructive actions, external-impact actions, credential handling, publishing, deployment, or broad architecture changes.

The chosen mode should be recorded in `AGENTS.md` and `.agent/memory/active-context.md`.

> Regardless of mode, the AI should always ask before deleting data or files, overwriting user work, changing git history, publishing or deploying, handling real secrets, or making broad architecture changes.

8. Closeout Protocol

The closeout is not optional. Every session that ends without it puts continuity at risk. It takes 2 minutes — the AI handles it, you just trigger it.

1 **Trigger the closeout**

Say: **"Run the closeout protocol."**

2 **The AI executes 7 steps**

Step	Action	What Gets Updated
1	Summarize completed work	Lists 2–5 specific things finished this session
2	Update active-context.md	New current state, next step, any blockers
3	Append to changelog.md	Dated entry summarizing the session's changes
4	Update in-progress.md	Marks completed tasks, updates status of active ones
5	Move to completed.md	Finished tasks moved with today's date
6	Add to lessons-learned.md	Any new rules, patterns, or discoveries worth keeping
7	Write next session steps	2–4 specific next actions added to active-context.md

3 **The Mandatory Memory Test**

After the 7 steps, the AI runs this check before the session can close:

> *"If a new AI session started right now and read only the startup files, would it know exactly where this project is and what to do next?"*

If the answer is YES: session is complete. If the answer is NO: the AI updates the missing files and repeats the test. The session cannot close until the answer is YES.

9. Checkpoint Protocol

A Checkpoint saves your session's progress mid-flight without ending the session. Use it when a session is getting long, before a risky change, or any time you want to protect work against context compacting.

1 **Trigger the checkpoint**

Say: **"Run checkpoint."**

2 **The AI executes 3 steps**

Step	Action	What Gets Updated
1	Update active-context.md	Current phase, what was just done, immediate next steps
2	Update in-progress.md	Moves completed sub-tasks to "Just Completed (This Session)"
3	Append to changelog.md	One-line note: date + brief summary of progress so far

3 **Continue working**

The AI confirms *"Checkpoint saved."* and the session continues. A Checkpoint does not end the session.

Checkpoint vs Closeout: Checkpoint saves progress mid-session — 3 steps, 30 seconds. Closeout ends the session with a complete memory update — 7 steps, 2 minutes. A session with multiple Checkpoints still requires a full Closeout at the end.

10. New Project Workflow

Use this path when starting a brand-new project from scratch.

1 **Run the initialization script**

From the parent directory where the project should be created:

```
bash ~/Tools/Alubys-AI-Dev-System/scripts/init-new-project.sh my-project-name
cd my-project-name
```

This creates the project folder with `AGENTS.md`, `CLAUDE.md`, all mode prompt files, and the full `.agent/` memory structure.

2 **Open the project in your AI tool**

For file-aware agents (Claude Code, Cursor, Codex), open the project folder. The AI will read `AGENTS.md` automatically. For chat-only tools, open the project folder in your text editor.

3 **Load the session**

For file-aware agents, say: *"Follow AGENTS.md and use _PROMPTS/START_NEW_PROJECT_PROMPT.md to start this project."* For chat-only tools, paste `_PROMPTS/MASTER_SYSTEM_PROMPT.md` then `_PROMPTS/START_NEW_PROJECT_PROMPT.md`.

4 **Choose your Approval Mode**

Before work begins, tell the AI which approval mode this project will use: Plan-First, Standard, or Autonomous.

5 **Begin Brainstorming**

Say: *"I want to start a new project. Let's begin with brainstorming."* The AI will ask about goals, users, constraints, and scope.

> **Important:** Do not skip Brainstorming. Most AI-assisted projects fail because users jump from idea to code. The structured phases exist to prevent this.

11. Existing Project Workflow

Use this path when adding the Alubys system to a project that already exists.

> **Back up first.** Before running any script on an existing project, make a backup. Duplicate the project folder in Finder, or commit all changes to git before proceeding.

1 Navigate to your project and run the copy script

```
cd ~/Projects/my-existing-project
bash ~/Tools/Alubys-AI-Dev-System/scripts/copy-agent-system.sh
```

The script adds only new files — it never overwrites or modifies anything you already have.

2 Review what was added

The script creates `AGENTS.md`, all mode prompt files, and the `.agent/` memory structure. If `README.md` already existed, a `README.ToBeMerged.md` is created — review it and manually merge any useful sections.

3 Load the session

For file-aware agents: *"Follow AGENTS.md and use _PROMPTS/ONBOARD_EXISTING_PROJECT_PROMPT.md to onboard this project."* For chat-only tools, paste `_PROMPTS/MASTER_SYSTEM_PROMPT.md` then `_PROMPTS/ONBOARD_EXISTING_PROJECT_PROMPT.md`.

4 Let the AI inspect — do not interrupt

The AI reads the codebase, identifies the tech stack, summarizes the architecture, and flags technical debt. Let it complete before you say anything.

5 Review and correct the generated documentation

The AI drafts `architecture-proposals.md`, `active-context.md`, `roadmap.md`, and `backlog.md`. Review each one — correct anything wrong and add business context the AI could not infer from code.

6 Choose your Approval Mode and starting workflow mode

Tell the AI which approval mode this project will use, and which workflow mode fits right now: Brainstorming, Planning, Execution, or Reflection.

12. Git And Memory Policy

Commit `.agent/` by default, including `.agent/sessions/`.

The `.agent/` directory is durable project memory. If it is ignored, the project can lose continuity across machines, contributors, or system failures.

The one exception is local secrets:

```
.agent/secrets.local.md
```

That file is ignored by `.agent/.gitignore` and must never be committed.

13. Secret Handling

Tracked files may reference secret names, but must never contain real secret values.

Good examples:

- `Uses OPENAI_API_KEY from the deployment environment`
- `DATABASE_URL is configured in the hosting provider`
- `Local-only setup notes are in .agent/secrets.local.md`

Bad examples:

- Actual API keys
- Passwords
- Tokens
- Private certificates
- Secret-bearing screenshots or logs

Real values belong in environment variables, a password manager, deployment secret storage, or `.agent/secrets.local.md`.

14. Templates And Scripts

The installer scripts generate all project files from the `templates/` directory. You do not need to modify templates to use the system.

Mac / Linux: `init-new-project.sh` and `copy-agent-system.sh`

Windows: `init-new-project.ps1` and `copy-agent-system.ps1` — native PowerShell equivalents, no Git Bash or WSL required.

```
# Windows — new project
.\scripts\init-new-project.ps1 my-project-name

# Windows — add to an existing project
.\scripts\copy-agent-system.ps1 C:\path\to\existing-project
```

> **Windows first-time setup:** If PowerShell blocks the script, run this once in PowerShell: `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`

If you want to customize the files that get installed into your projects — for example to pre-fill your organization name, standard tools, or default approval mode — edit the corresponding files in `templates/` before running the installer scripts.

The `templates/` directory contains the source for every file that gets installed: `AGENTS.md`, `CLAUDE.md`, `README.md`, and all `.agent/` memory file stubs.

15. Memory Files Reference

All project memory lives in the `.agent/` directory inside your project.

File	Purpose	Updated When
`memory/active-context.md`	Current state — mode, phase, next step, blockers	End of every session
`memory/decisions.md`	Architecture and technical decisions with rationale	A decision is made
`memory/lessons-learned.md`	Rules and patterns discovered during the project	After mistakes or discoveries
`memory/changelog.md`	Session-by-session log of what changed	End of every session
`planning/brainstorming.md`	Exploratory notes from brainstorming phase	During brainstorming
`planning/product-specification.md`	Definitive scope and MVP definition	During planning
`planning/architecture-proposals.md`	Tech stack, architecture, data model	During planning
`planning/constraints.md`	Hard limits: timeline, budget, tech, compliance	Brainstorming / planning
`planning/risks.md`	Known risks and mitigation strategies	Brainstorming, revisit in reflection
`planning/implementation-phases.md`	Phased build plan with tasks per phase	During planning
`tasks/roadmap.md`	High-level milestones and project direction	At phase completion
`tasks/backlog.md`	All known tasks not yet in progress	Continuously
`tasks/in-progress.md`	Active tasks only — max 5–7 items	Every session
`tasks/completed.md`	Append-only record of finished work	When tasks complete
`sessions/README.md`	Log of AI work sessions	End of every session
`reflections/weekly-review.md`	Periodic project health review	Weekly or at phase end
`reflections/agent-improvements.md`	Observations about AI behavior to improve	During reflection

Memory Rules

- Store: architecture decisions, recurring bugs, deployment caveats, constraints, user preferences
- Never store: secrets, API keys, passwords, transcript dumps, temporary debug output
- Size rule: when any file exceeds ~200 lines, compress it during Reflection Mode

16. Quick Reference Card

How to "Paste a File"

> Open the `.md` file from `_PROMPTS/` in VSCode → Select All (Cmd+A on Mac, Ctrl+A on Windows) → Copy (Cmd+C / Ctrl+C) → Paste into AI chat (Cmd+V / Ctrl+V). Always paste the raw text — do not attach as a file.

Session Startup — 30 seconds

```
1. Open a new session with your AI assistant
2. For chat-only tools: paste _PROMPTS/MASTER_SYSTEM_PROMPT.md
3. Say: "Read the startup files and tell me where we are."
4. AI reads 5 files, gives status summary, work begins
```

Session Closeout — 2 minutes

```
Say: "Run the closeout protocol."
AI runs 7 steps, then runs the mandatory Memory Test.
Session only closes when Memory Test passes.
```

Mid-Session Checkpoint — 30 seconds

```
Say: "Run checkpoint."
AI saves active-context.md, in-progress.md, and changelog.md.
Work continues — session does not end.
Use on long sessions or before risky changes.
```

Which Prompt File to Use

Situation	Use This File
Any session start	`_PROMPTS/MASTER_SYSTEM_PROMPT.md`
First session, new project	`_PROMPTS/START_NEW_PROJECT_PROMPT.md`
First session, existing project	`_PROMPTS/ONBOARD_EXISTING_PROJECT_PROMPT.md`
Exploring a problem	`_PROMPTS/BRAINSTORMING_MODE_PROMPT.md`
Designing architecture	`_PROMPTS/PLANNING_MODE_PROMPT.md`
Building features	`_PROMPTS/EXECUTION_MODE_PROMPT.md`
Weekly review / phase end	`_PROMPTS/REFLECTION_MODE_PROMPT.md`

Common Mistakes to Avoid

Mistake	Why It Hurts
Skipping startup protocol	AI starts cold and gives irrelevant suggestions
Skipping closeout protocol	Session work evaporates — next session rediscovers it
Skipping Checkpoint on long sessions	Context compacting mid-session can lose in-progress reasoning
Attaching .md files instead of pasting	Some AI interfaces don't read attachments the same way
Cloning the repo into every project	Clone once to ~/Tools — use the scripts
Not backing up before onboarding	Copy script is safe, but always back up live projects first
Jumping from idea to code	Use Brainstorming and Planning modes first
Storing secrets in files	Never — use environment variables only
Skipping Reflection Mode	Technical debt accumulates silently — run it weekly

17. Legal

This software is provided under the MIT License. See [LICENSE](./LICENSE) for the full text.

By using this system in any form, you agree to the [Terms of Use](https://alubysllc.github.io/terms). Use is entirely at your own risk. Alubys, LLC is held harmless from any claims arising from use of this system, including data loss, AI token costs, and any consequences of AI-generated content.

See [DISCLAIMER.md](./DISCLAIMER.md) for a summary of limitations.

> **Click-wrap:** The installer scripts (`init-new-project.sh`, `copy-agent-system.sh`, `init-new-project.ps1`, `copy-agent-system.ps1`) display the Terms of Use and require typing `YES` before proceeding. This constitutes acceptance of the Terms of Use.

18. Support This Project

The Alubys AI Development System is provided as a free public resource. If it helps you preserve AI memory, organize project work, or work more effectively with AI agents, you are welcome to support continued development and related public tools.

Support can be sent through Zelle by scanning this QR code for **ALUBY'S LLC**: